

EXECUTIVE SUMMARY

HORIZON GRID

Executive Overview

Version: 0.2.4

Documentation prepared: 2026-08-21

PREPARED FOR EVALUATION

Table of Contents

Executive Summary	3
The Problem and The Solution	5
Feature Inventory	7

Executive Summary

HORIZON GRID is a self-hosted threat-intelligence platform. An analyst gives it one indicator of compromise — an IP address, domain, URL, file hash, or CVE ID — and it queries 18 independent intelligence sources at once, correlates what they say about each other, computes a deterministic threat score from that evidence, and has an AI model explain the result in plain language, with every claim traceable back to the real data behind it. The product's own tagline states the design goal directly: **Every Signal. One Operational Picture.**

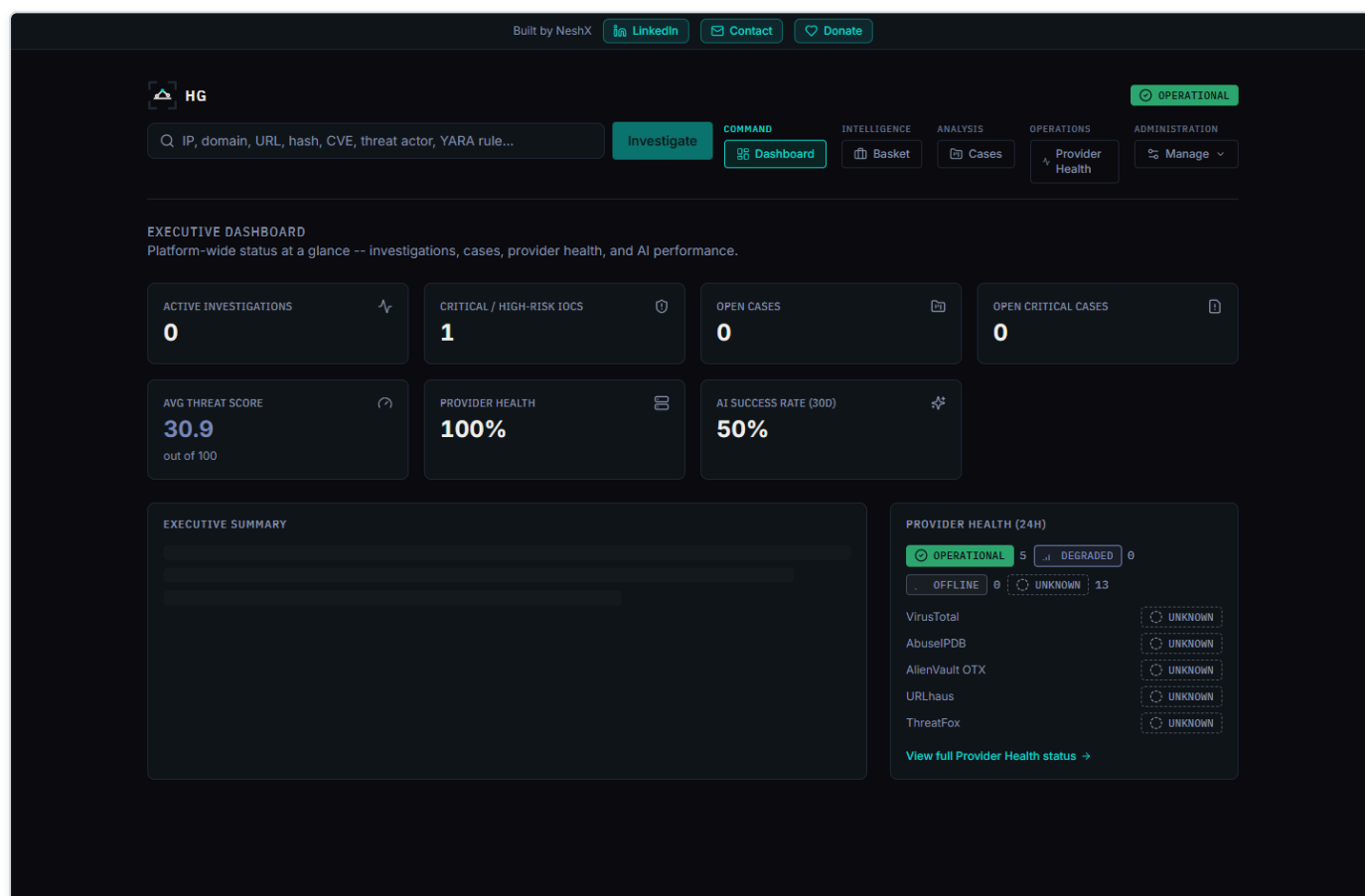


Figure 1 — The Executive Dashboard — every KPI tile is a real query result against the live database, never a hardcoded display value.

What Problem Does It Solve?

No single threat-intelligence source sees everything. A file hash flagged by one antivirus engine might be ignored by another; an IP address on one blocklist might look unremarkable on the next. Getting a trustworthy read on an indicator normally means an analyst manually checking it against several independent sources, one browser tab at a time, and holding all of it in their head well enough to notice when two sources are describing the same underlying fact from different angles. That manual cross-referencing is the real cost of threat triage — and it is exactly the cost that gets skipped when the queue of indicators is long.

What Does HORIZON GRID Do About It?

It collapses that workflow into a single search box. One submission fans out to every applicable provider in parallel; results correlate automatically; a fixed, versioned, non-AI scoring engine turns the combined evidence into a threat score before any AI is involved; and an AI model is then handed that score as a given fact to narrate, not a number it's free to invent. The platform tracks its own AI outcomes honestly — a real success, a correct decision to skip the AI when there's no evidence to reason over, or a disclosed failure with a deterministic fallback — never a fabricated result.

Who Uses It?

A solo analyst can run the whole stack on one Windows or Linux machine behind a guided installer; a small security team can point several browsers at the same install and share cases, a shared watchlist ("the Basket"), and one live operational dashboard. Three fixed roles — Admin, Analyst, Viewer — are enforced on every backend route, not just hidden in the UI, so a broader audience (including a Viewer who should see the operational picture but not run scans or export data) can be given access safely.

What Makes It Different?

Two things a typical dashboard tool doesn't do:

1. **The threat score is computed before the AI ever sees it, not by the AI.** A deterministic scoring engine (versioned, currently 1.0) turns provider-verdict consensus and correlation-graph evidence into overall_risk_score, confidence_score, malicious_probability, and a severity band using a fixed, documented formula. The AI is handed that number as an input and narrates it — the platform re-validates the AI's own stated verdict against that number before saving anything, so the AI cannot quietly override the math.
2. **Every AI claim is checkable against real evidence, not just plausible.** An Evidence Ledger and a set of verdict-interrogation tools ("Why?", "Challenge This Verdict", "False Positive Check", "Score Explanation", "Intelligence Conflicts") let an analyst check any AI statement against the concrete, non-AI-generated facts the platform actually collected — rather than trusting a well-written paragraph on faith.

Why It Matters

Analysts don't need another tool that produces a confident-sounding paragraph about an indicator — they need one that shows its work. HORIZON GRID's central design bet is that a security tool earns trust by being *auditable*: a fixed score computed the same way every time, an AI that narrates rather than decides, and a health/operational picture (Provider Health, Executive Dashboard) that is never allowed to show a hardcoded or invented number. That bet shaped every architectural decision documented in the rest of this submission.

The Problem and The Solution

The Problem: Fragmented, Manual Threat Triage

Every individual threat-intelligence source is narrow by design — a multi-engine reputation scanner, a malware sandbox, a certificate-transparency search, a government vulnerability catalog, a DNS blocklist — and none of them sees the full picture alone. In practice, investigating one indicator means an analyst opens a reputation lookup site, then a sandbox or sample database, then a certificate search, then a vulnerability database, then a blocklist, pasting the same value into each one and reading each result in its own format. Cross-referencing those results by hand — noticing that an IP tied to a malware family is also tied to a MITRE ATT&CK technique that matches a hash flagged somewhere else — is the actual cost of triage, and it is the first thing that gets skipped when the indicator queue is long.

A second, quieter problem sits underneath the first: once AI entered this workflow, it became tempting to let a language model produce the whole verdict from a prompt. That's fast, but it trades a real problem (manual cross-referencing is slow) for a worse one — an analyst now has to trust a paragraph that might be internally inconsistent, might invent a source that was never checked, and offers no fixed, reproducible number to audit later.

The Solution: One Submission, One Verifiable Operational Picture

HORIZON GRID's actual processing pipeline, in the order it really executes:

1. **IOC submission and classification** — the analyst submits one value; the platform validates and classifies it (IPv4/IPv6, domain, URL, file hash, CVE ID, and the other IOC types the app recognizes).
2. **Parallel provider queries** — every one of the 18 built-in providers relevant to that IOC type is queried at once (a file hash is never sent to a domain-registration lookup), and results stream into the page live over Server-Sent Events as each provider responds, rather than waiting for the slowest one.
3. **Per-provider AI summaries** — as each provider's result arrives, a short AI-written summary of that one result is generated, grounded only in that provider's own data.
4. **Correlation** — once providers have reported in, a correlation engine extracts relationships between what different sources said (shared infrastructure, related hashes, malware families, MITRE ATT&CK techniques, exploited CVEs) and boosts confidence when independent providers corroborate the same fact.
5. **Deterministic threat scoring — before the AI's final verdict, not after.** A fixed, versioned scoring engine computes `overall_risk_score`, `confidence_score`, `malicious_probability`, and a severity band from provider-verdict consensus and correlation evidence, using a documented, weighted formula (see the Threat Scoring guide). This step deliberately happens *before* the AI's consolidated assessment.
6. **AI Final Assessment, conditioned on the score** — one consolidated AI pass reasons over every per-provider summary plus the correlation output, and is handed the already-computed score as a fixed input to narrate. The platform validates that the AI's own stated verdict agrees with that number before saving anything, rather than trusting the AI to have applied the number correctly.
7. **Operational presentation** — the result renders as an investigation page with an Evidence Ledger and verdict-interrogation tools, rolls up into the Executive Dashboard's live KPIs, and can be exported (JSON/Markdown/CSV/PDF) with the platform's branding, a generation timestamp, and an Investigation ID on every export.

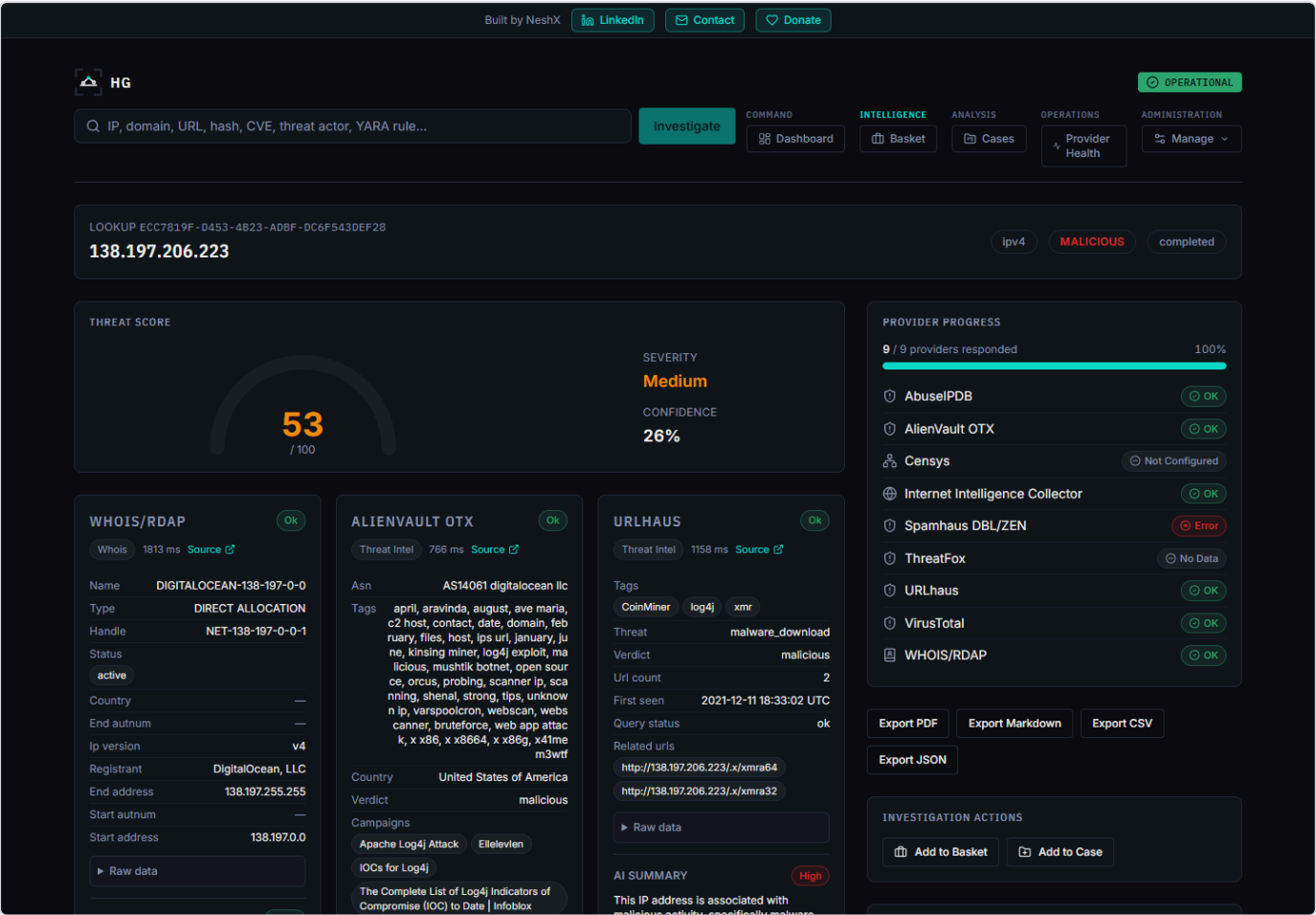


Figure 2 — A completed investigation — provider results, threat score, and severity all visible on one page, each traceable back to the specific source that produced it.

The order in step 5 and step 6 is the load-bearing design decision in this whole pipeline: the AI is never the source of the threat score. It receives a number that was already computed by a fixed formula and is only permitted to explain it — which is what makes the score reproducible and auditable in a way a single AI-generated verdict never can be.

Why This Design, Specifically

An earlier, simpler version of this idea would have let the AI produce `risk_score` , `malicious_probability` , and the verdict directly from a prompt in one call. That's a real, common pattern, and it's exactly the pattern this project moved away from once a deterministic scoring engine was built: prompt wording alone cannot reliably stop a model — particularly a local model — from overriding a "do not fill in, this is provided" instruction. Computing the score first and feeding it into the AI prompt as a fixed input, then validating the AI's verdict against it, closes that gap structurally instead of relying on the AI to behave. It is also the only design that makes an "average threat score" KPI on the Executive Dashboard mean anything consistent over time, since the AI backend an analyst has selected (of eleven interchangeable options) no longer changes what the score is — only how it's explained.

Feature Inventory

Every item below is implemented and verified against current source code (`backend/app/providers/registry.py` , `backend/app/core/runtime_config.py` , `backend/app/scoring/engine.py` , `backend/app/models/user.py` , `backend/app/security_assessment/`) — nothing here is planned or aspirational. Deep-dive detail on each area lives in the companion technical documentation set (Provider Guide, AI Guide, Threat Scoring guide, Admin Guide, Security guide) referenced at the end of this submission.

IOC Investigation

The core workflow: submit one indicator (IP, domain, URL, file hash, CVE, and 27 other recognized `IOCType` values — 32 in total), and the platform validates, classifies, and fans the value out to every registered provider that supports that type. Results stream in over Server-Sent Events rather than blocking on the slowest provider.

Provider Ecosystem — 18 Providers

`VirusTotal` , `AbuseIPDB` , `AlienVault OTX` , `URLhaus` , `ThreatFox` , `MalwareBazaar` , `crt.sh` , `NIST NVD` , `CISA KEV` , `MITRE ATT&CK` , `WHOIS/RDAP` , `urlscan.io` , `Google Safe Browsing` , `Hybrid Analysis` , `Spamhaus` , `PhishTank` , `Censys` , and the platform's own **Internet Intelligence Collector** (a live OSINT crawler) — spanning multi-engine reputation scanning, community abuse/malware databases, certificate-transparency search, government vulnerability catalogs, sandbox behavior, DNS blocklists, phishing databases, and internet-wide host scanning. Several (`crtsh` , `cisa_kev` , `mitre_attack` , `whois_rdap` , and the stub-implemented `spamhaus` / `phishtank`) need no API key at all and are active from first boot.

AI System — 11 Interchangeable Backends

`Ollama` (local, free, no key), `Anthropic` , `AWS Bedrock` , `Google Gemini` , `Groq` , `OpenAI` , `Kimi` (Moonshot AI), `DeepSeek` , `xAI` (Grok), `Mistral AI` , and `OpenRouter` — all exposed through one identical `call_claude_json()` interface, switchable at runtime with no restart. Two distinct AI passes run per investigation: a grounded per-provider summary as each result arrives, and one consolidated Final Assessment once every provider has reported, conditioned on the deterministic threat score (see below). Every AI generation is tracked as one of three honest outcomes — real success, correct no-evidence skip, or disclosed failure — never a fabricated result.

Deterministic Threat Scoring

A fixed, versioned (`SCORING_ENGINE_VERSION = "1.0"`) scoring engine computes `overall_risk_score` , `confidence_score` , `malicious_probability` , and a severity band from provider-verdict consensus and correlation-graph evidence — **before** the AI's final assessment runs, not after. The AI receives the score as a fixed input and is validated against it, so it narrates rather than decides. A dedicated corroboration-scaling mechanism prevents one account listing several tags across free-text fields from single-handedly flooding the score.

Correlation and Evidence

A pure, deterministic correlation engine extracts typed relationship edges (shared infrastructure, related hashes, malware families, MITRE ATT&CK techniques, exploited CVEs) from normalized provider fields and boosts confidence when independent providers corroborate the same fact. A non-AI-generated Evidence Ledger, built directly from provider results and correlation edges, backs a set of verdict-interrogation tools (WHY malicious, What Is This, Provider Disagreement, False-Positive Assessment, Challenge/red-team, Smart Next Actions, Intelligence Gaps, Score Explanation) plus Copilot Q&A, IOC comparison, and hunting-query/detection-rule generation.

Executive Dashboard

Live KPI tiles — active investigations, critical/high-risk IOC count, open case counts, average threat score, provider health percentage, AI success rate — computed from real database aggregates, never hardcoded, plus an AI-generated narrative that is honestly labeled as AI-generated or template-fallback.

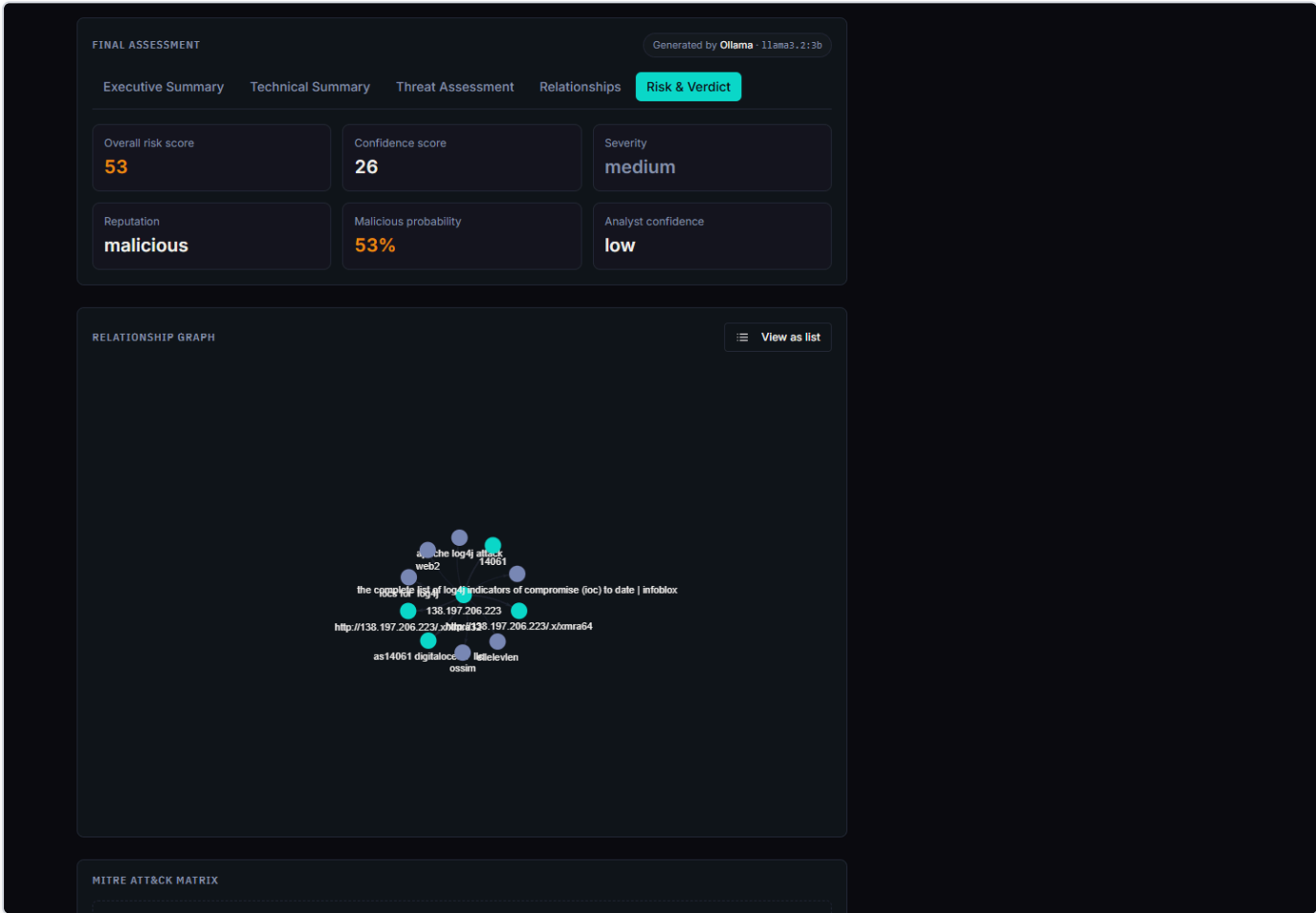


Figure 3 — The Final Assessment panel's Risk & Verdict tab — overall risk score, confidence score, severity, and malicious probability, all computed before the AI narrates them.

Provider Health

A dedicated page tracking every registered provider's real status (Healthy/Degraded/Down/Unknown) across four rolling windows (1h/24h/7d/30d), built from the same `ProviderResultRecord` data every real investigation already writes. A provider with zero attempts in a window shows Unknown, never a false Healthy.

Security Assessment Toolkit (Active Port Scanner)

A genuinely active check — Nmap port/service scan, DNS record lookup, TLS certificate inspection, HTTP security-header check — against a target the analyst explicitly confirms authorization for (target retype + authorization checkbox), with real cancellation support (a live task can be stopped mid-scan, not just flagged in the database). Findings feed the investigation's risk/confidence floor without inflating `malicious_probability` on their own.

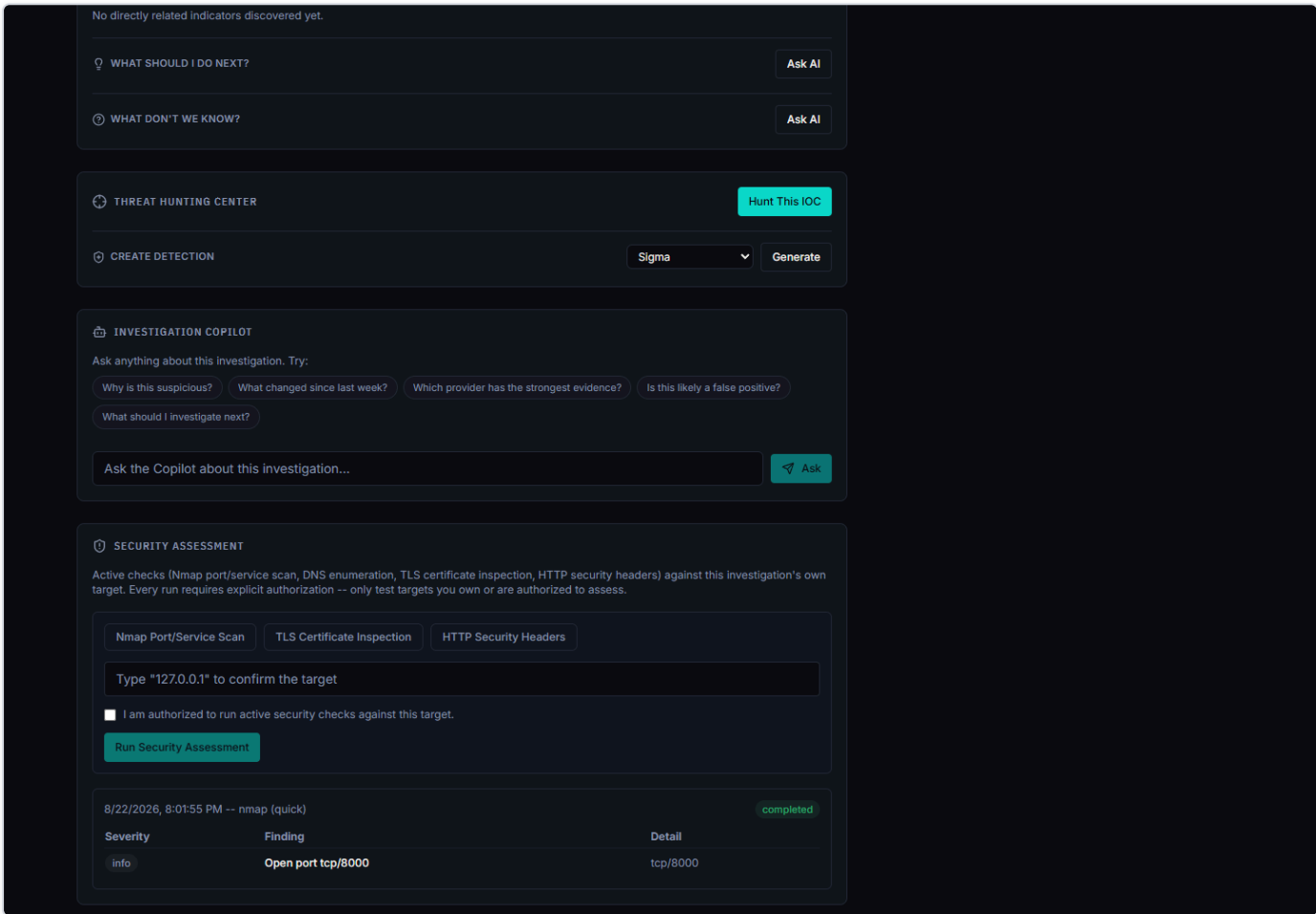


Figure 4 — A completed Nmap "quick" scan against an authorized target, showing the required target-confirmation/authorization controls and a real finding.

Case Management and IOC Basket

Group indicators into a case with analyst notes as an incident develops, or keep a running personal watchlist of indicators worth revisiting.

Administration and RBAC

Three fixed roles — Admin, Analyst, Viewer — enforced server-side on every route via a `require_permission()` dependency, not just hidden in the UI. Genuine multi-administrator support (any admin can create further admin accounts from the console; the platform does not hardcode a single superuser). A full audit log records configuration and security-relevant changes.

Exporting Findings

JSON, Markdown, CSV, and PDF export, gated behind an export-specific permission distinct from ordinary read access. CSV and PDF paths carry tested defenses against formula-injection and markup-injection abuse. Every export carries the platform's branding, a generation timestamp, and an Investigation ID.

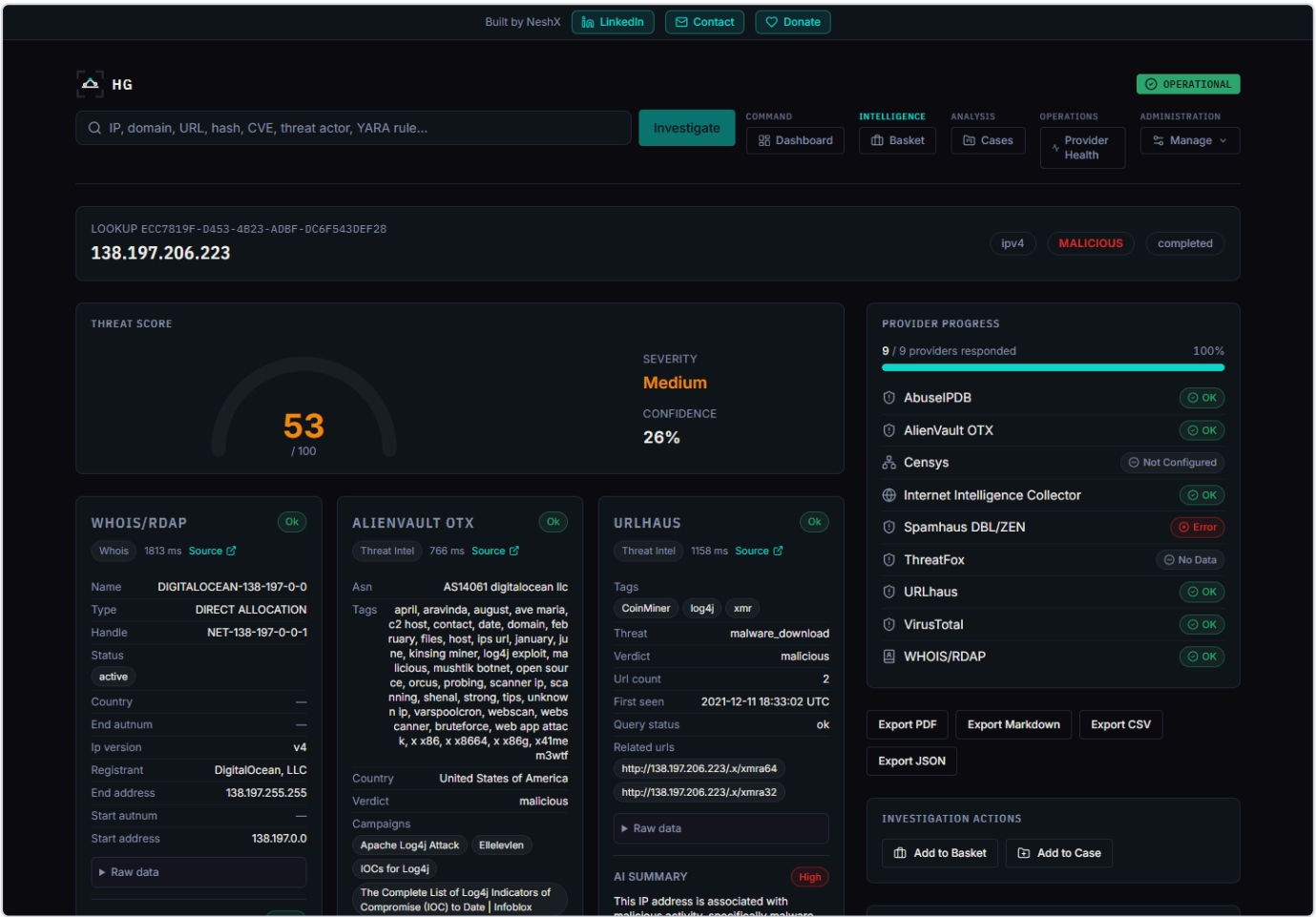


Figure 5 — The export panel on a completed investigation — JSON, Markdown, CSV, and PDF, each carrying platform branding and an Investigation ID.

Deployment

A Windows Inno Setup installer with a guided WinForms Setup Wizard (administrator account, AI backend, provider credentials with live "Test" buttons, network ports, install), and a Linux `.deb` package with an equivalent setup flow — both running the actual application as Docker Compose, plus a parallel Kubernetes/Kustomize deployment alternative for either platform.